



ADAPT

A Framework for Agile Distributed Software Development

Raoul Vallon, Stefan Strobl, Mario Bernhart, Rafael Prikladnicki, and Thomas Grechenig

AGILE DEVELOPMENT and distributed software development (DSD) seem like an impossible match. Agile processes are designed for collocated teams collaborating closely, which appears incompatible with DSD and its inherent coordination, control, and communication challenges.¹ Nonetheless, 82 percent of respondents to VersionOne Inc.'s 10th Annual State of Agile survey, conducted in 2015, said they had at least some distributed teams practicing agile development, a significant rise from only 35 percent just three years earlier.²

To gather evidence about the past and current state of research on teams using both agile and distributed development, three of us conducted a systematic literature review of the topic covering 1999 to 2014. (For the review protocol, see the Web extra at <https://extras.computer.org/extra/mso2016060106s1.pdf>.) We found an increasing interest in applying agile practices to DSD from 2004 in the form of anecdotal experience reports and from 2010 in the form of evaluation papers, mainly case studies that indicated a switch to more rigorous research approaches.

However, past research seldom fully reported contextual details, omitting information such as team size, thus limiting the results' generalizability. In response, we created a checklist for

fully reporting the empirical context of case studies on agile DSD, as well as general information about projects. (For the checklist, see the Web extra at <https://extras.computer.org/extra/mso2016060106s2.pdf>.)

Also, although past research reported lessons learned in agile DSD, no one has created a comprehensive framework describing how to apply agile practices to DSD to which both researchers and industrial practitioners can relate. We launched our multiyear research project primarily to create such a framework—ADAPT (Agile Distributed Adaptable Process Toolkit).

Evidence from a Multiple-Case Study

We studied multiple cases to generate ADAPT's empirical base. We included selected Scrum-based cases to present the state of the industry because we found that most reported cases in the past 15 years used either Scrum or Scrum with Extreme Programming. We selected the following heterogeneous, information-rich cases³ to produce stronger results:⁴

- Case CrossTown—two development sites in the same city (Vienna)
- Case NoTimeshift—two sites in the same country (Austria)

TABLE 1

Contextual information about the three cases in the ADAPT (Agile Distributed Adaptable Process Toolkit) study.

Topic	Contextual details	Case		
		CrossTown	NoTimeshift	Continental
Distribution details	Location	Onshore	Onshore	Offshore
	Insourcing or outsourcing	Insourcing	Outsourcing	Insourcing
	Geographic distance	Near	Near	Near
	Time-zone difference	None	None	Small
	Sociocultural distance	None	None	Low
	Supplier location	Austria	Austria	3 European countries
	Customer location	Austria	Germany	1 European country
	No. of sites	2	2	3
	Team distribution	Integrated teams	Integrated teams	Isolated teams
Project details	Domain	Web and hardware	Enterprise software	Web
	Experience with distributed software development	10 years	2 years	15 years
	Team size	Overall: 19 Site 1: 13 Site 2: 6	Overall: 30 Site 1: 20 Site 2: 10	Overall: 39 Site 1: 14 Site 2: 19 Site 3: 6
	Duration	15 mos.	6 mos.	9 mos.
	Type	Industry	Industry	Industry
	Successful	Yes	Yes	Yes
Agile details	Agile experience	7 years	3 years	3 years
	Agile process	Scrum	Scrum	Mixed
	Agility level	All teams	All teams	All teams

- Case Continental—three sites in the same continent (Europe)

Our checklist (see Table 1) drove our data collection to ensure reporting of a complete contextual picture. For each case, we first conducted an analysis resulting in sets of candidate guidelines for eliminating the challenges that agile DSD faces (15

derived from CrossTown, 22 from NoTimeshift, and 12 from Continental) and recommended candidate development-team practices (40 from CrossTown, 35 from NoTimeshift, and 19 from Continental).

We combined the candidate guidelines and practices to create empirically stronger versions by either identifying duplicates or com-

binning those that address variations of the same basic theme. We then linked practices to guidelines they could help implement and linked each guideline to the one challenge it could help mitigate. Table 2 shows these linkages. (For a schematic of the case study design, see the Web extra at <https://extras.computer.org/extra/mso2016060106s3.pdf>.)

TABLE 2

In ADAPT, the guidelines (G1–G10) are linked to the challenges they mitigate.

Practice	Coordination challenges			G4. Use a retrospective as a driver for continuous improvement.
	G1. Strive for all sites' equal involvement.	G2. Let long-lived cross-functional teams self-organize.	G3. Establish knowledge and information flow between sites.	
P1. Traveling ambassador	✓		✓	
P2. Full-team onsite sessions			✓	
P3. Team rotations	✓		✓	
P4. Team events		✓		
P5. Scrum master on each site	✓			
P6. Cross-site reviews				
P7. Multiway informal communication		✓	✓	
P8. Meeting minutes			✓	✓
P9. Ad hoc screen sharing				
P10. Synchronized sprints	✓			
P11. Accessible backlogs	✓			
P12. Tangible requirements (behavior-driven development)			✓	
P13. Customer requirements workshop			✓	
P14. Feature-team organization		✓		
P15. Multiteam workflow board				
P16. Multilevel onsite proxy planning	✓			
P17. Separation of roadmap and sprint planning	✓		✓	
P18. Onsite retrospective with proxies				✓
P19. Onsite sprint review with proxies				
P20. Establish metrics to evaluate the process				✓
P21. Daily intrateam communication		✓		
P22. Interteam scrum of scrums		✓		
P23. Frequent deployments to customer				
P24. On-demand specification meetings	✓			
P25. Cross-team specialist meeting		✓		
P26. Global all-site broadcast meeting	✓		✓	
P27. User story requirements			✓	
P28. Code quality and standards			✓	
P29. Agile coach				

* A checkmark means the indicated practice helps implement the guideline.

Control challenges			Communication challenges			
	G5. Truthfully visualize all sites' workflow.	G6. Invite the customer in with frequent releases.	G7. Embrace quality over feature rush.	G8. Build a tool chain to support agile practices.	G9. Establish multilevel, multiconcern feedback loops.	G10. Step by step, create an environment in which agile can work.
	✓					✓
	✓		✓		✓	
				✓		✓
				✓	✓	
	✓			✓		
	✓	✓				
		✓				
	✓			✓		
					✓	
			✓		✓	
	✓			✓		
					✓	
					✓	
		✓	✓	✓		
			✓			
					✓	
		✓				
			✓			
						✓

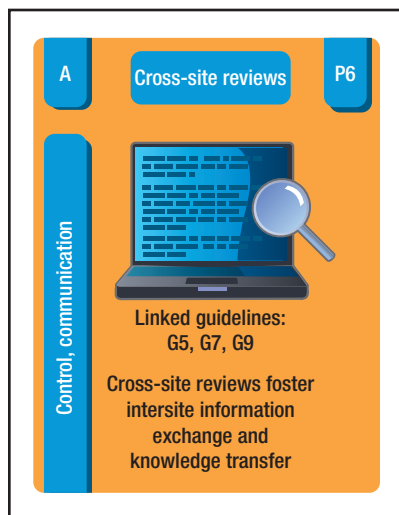


FIGURE 1. This practice card illustrates practice P6, “Cross-site reviews,” in which developers at one site review code or other artifacts such as test cases from another site. This helps transfer knowledge between sites and improves software quality. Each card shows the practice identifier (in this case, P6), title (Cross-site reviews), the challenges the practice mitigates (control, communication), the guidelines it helps implement (G5, G7, G9), a one-line description, and an icon.

Evolving the Method

For practitioners, ADAPT provides in-depth descriptions of each practice and guideline. For researchers, it thoroughly documents how we developed the framework. As part of this effort, we designed practice cards (Figure 1 shows an example) to be used in kickoff workshops and retrospective meetings in which developers work on practices critical to satisfying specific guidelines.

We completed ADAPT’s design theory⁵ during coauthor Raoul Vallon’s four-month visit to Stanford University’s Center for Design Research, and we plan future iterations. Five testable propositions

(TPs) describe the nature of current and future iterations:

- TP1—The framework’s guidelines and practices are grounded in empirical evidence.
- TP2—The framework allows simple, pragmatic, and iterative process tailoring.
- TP3—The framework supports project-based process tailoring, making it useful in different situations.
- TP4—The framework provides tangible, detailed advice to practitioners.
- TP5—The framework is easily extensible for future modifications.

The propositions enable the framework to scale while remaining true to its origins and intended design.

Using ADAPT

ADAPT lets developers tailor the process implementation step-by-step to a project’s constraints. This capability also lets teams cope with changing constraints throughout a project’s life, such as a development site’s addition or removal.

Table 2 shows the full ADAPT framework with the three categories of challenges spanning 10 guidelines and 29 practices. Developers can use it when they set up a new process implementation for a project in a DSD environment or add it to ongoing efforts. They can use it to tailor the selection of agile practices to project constraints via a continuous three-step cycle.

First, developers refer to the ADAPT guidelines to find suitable linked practices to add for the next sprint iteration to address all the guidelines. A new process implementation should have at least one prac-

tice per guideline. It’s better to use fewer practices per guideline at the beginning of the process.

Second, the developers conduct the sprint iteration and then collect data to evaluate the practices’ effectiveness via, for example, burn-down charts, bug counts, or lead time.

Third, the developers evaluate the data and reflect on the process implementation as a team during, for example, a retrospective. Then, they start over with the first step but now consider adding, modifying, or removing practices while trying to balance the coordination, control, and communication challenges.

We evaluated the current guidelines and practices via interviews with 10 industrial and academic experts from Austria, Finland, Germany, and the US. Table 3 shows the results. The response was positive. We’re still working on several aspects of the framework such as scalability, usability, and practitioner utility.

We regard ADAPT as an important step in creating a common foundation for implementing agile DSD projects. We invite practitioners to test it and provide feedback, and we ask researchers to help extend the framework. 

References

1. E. Carmel and R. Agarwal, “Tactical Approaches for Alleviating Distance in Global Software Development,” *IEEE Software*, vol. 18, no. 2, 2001, pp. 22–29.
2. *10th Annual State of Agile Report*, VersionOne, 11 July 2016; <http://stateofagile.versionone.com>.
3. M.Q. Patton, *Qualitative Research and Evaluation Methods*, Sage Publications, 2002.

TABLE 3

Feedback on ADAPT from 10 interviews with industrial and academic experts.

General interviewee response	Issue	Summarized feedback	Interviewees
Approval	Testable propositions and research approach	ADAPT was adaptable, easily extensible, tangible, and detailed. The case study approach fit the problem well.	1, 6, 7, 8, 9
	Detailed practices	Practices were regarded as the central component. The guidelines provided good meta-information that was ready to use in an implementation study.	2, 5, 6, 7, 8, 9, 10
	Overview table	The table was well designed and had all necessary information on one page.	6, 7, 8, 9, 10
	Framework design	The modularity was good for future iterations. The hierarchy of challenges, guidelines, and practices was accessible and served the framework well.	2, 3, 5, 6, 7, 8, 9, 10
	Practice cards	The cards were a great idea, a good starting point, and worth testing in practice.	6, 7, 8, 9, 10
Improvements needed	Implementation study	Such a study would be very interesting. Interviewees 7, 9, and 10 showed interest in supporting the implementation in their industrial setting.	All
	Growing empirical basis	Once ADAPT's empirical basis is larger, it should be analyzed to determine whether the practices fit better with one of the distribution scenarios.	3, 6, 10
	Practice equivalency	Researchers should determine which practices are interchangeable, offering different perspectives on them such as simplicity, cost, and impact.	2, 5, 6, 7, 8, 9, 10

4. R. Prikladnicki, J.L.N. Audy, and J.R. Evaristo, "Distributed Software Development: Toward an Understanding of the Relationship between Project Team, Users, and Customers," *Proc. Int'l Conf. Enterprise Information Systems (ICEIS 03)*, 2003, pp. 417–423.

5. R. Vallon, *Empirically Driven Design of the Agile Distributed Adaptable Process Toolkit (ADAPT)*, tech. report, Austrian Marshall Plan Foundation, 2015; www.marshallplan.at/images/Vallon.pdf.

RAOUL VALLON is a postdoctoral researcher at TU Wien, where he leads the AMMA (Amazing Makers) work group on empirical software engineering. He developed the ADAPT (Agile Dis-

tributed Adaptable Process Toolkit) framework as part of his PhD research. Contact him at raoul.vallon@inso.tuwien.ac.at.

STEFAN STROBL is a PhD candidate and researcher with TU Wien's BUSY research team, which focuses on industrial software engineering and reengineering. Contact him at stefan.strobl@inso.tuwien.ac.at.

MARIO BERNHART is a software researcher and practitioner who leads TU Wien's BUSY software engineering research team. Contact him at mario.bernhart@inso.tuwien.ac.at.

RAFAEL PRIKLADNICKI is an associate professor at the Computer Science School and the director of the Science and Technology Park (Tecnopuc) at the Pontifical Catholic University

of Rio Grande do Sul, where he also leads the MuNDDoS research group. He's also on *IEEE Software's* editorial board. Contact him at rafaelp@pucrs.br.

THOMAS GRECHENIG is a professor of industrial software engineering and head of the Industrial Software Group at TU Wien. Contact him at thomas.grechenig@inso.tuwien.ac.at.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.